Реферат к докладу

Модель доступа AppArmor

Авраменко Давид Андреевич

Содержание

1 Введение **6**

2 Теоретические основы управления доступом **8**
2.1 Дискреционное управление доступом 8
2.2 Мандатное управление доступом 9
2.3 Назначение AppArmor в системе защиты Linux 9

3 Архитектура и принцип работы AppArmor **11**
3.1 Взаимодействие с Linux Security Modules 11
3.2 Профили приложений 12
3.3 Управление доступом на основе путей 13

4 Структура политики безопасности AppArmor **14**
4.1 Общий синтаксис профиля 14
4.2 Правила доступа к файлам 15
4.3 Контроль сетевого взаимодействия и полномочий 16
4.4 Абстракции, переменные и дочерние профили 17

5 Режимы работы и проверка политик **19**
5.1 Режим enforce 19
5.2 Режим complain 19
5.3 Состояние unconfined 20

6 Настройка и администрирование AppArmor в Linux **21**
6.1 Установка и проверка состояния 21
6.2 Создание и загрузка профиля 22
6.3 Формирование профилей на основе наблюдений 23
6.4 Диагностика отказов и анализ журналов 24

7 Применение AppArmor для защиты приложений **26**
7.1 Защита серверных служб 26
7.2 Рабочие станции, контейнеры и изоляция процессов 27

8 Сравнение моделей AppArmor и SELinux **28**

9 Преимущества, ограничения и рекомендации по использованию **30**
9.1 Преимущества AppArmor 30

9.2 Ограничения и возможные сложности 31
9.3 Рекомендации по безопасной эксплуатации 31

10 Заключение **33**

11 Список литературы **35**

Список иллюстраций

Список таблиц

1 Введение

Защита операционных систем является одной из основных задач информационной безопасности. В современных вычислительных средах одновременно работают пользовательские приложения, системные службы, веб-серверы, базы данных и средства удалённого администрирования. Каждая программа обращается к определённому набору ресурсов: файлам, каталогам, сетевым соединениям, устройствам и другим процессам. При наличии ошибки в программном обеспечении злоумышленник может попытаться использовать полномочия уязвимого процесса для получения конфиденциальной информации, изменения системных файлов или нарушения работы операционной системы.

Традиционная модель разграничения доступа в Linux основана на владельцах файлов, группах пользователей и разрешениях на чтение, запись и выполнение. Данная модель необходима, однако не всегда позволяет ограничить конкретное приложение в пределах его рабочих функций. Например, если служба запущена с правами пользователя, имеющего доступ к большому числу файлов, компрометация службы потенциально создаёт угрозу для всех доступных ей ресурсов.

Дополнительный уровень защиты обеспечивает **AppArmor (Application Armor)** — механизм мандатного управления доступом, реализованный на основе подсистемы Linux Security Modules (LSM). AppArmor задаёт допустимые действия программ с помощью специальных профилей безопасности и позволяет ограничивать доступ независимо от обычных разрешений

файловой системы [5; 8].

Цель реферата — изучить модель управления доступом AppArmor, рассмотреть архитектуру и принцип её работы, структуру профилей безопасности, способы настройки и диагностики, а также оценить преимущества и ограничения данного механизма.

Для достижения поставленной цели рассматриваются различия между дискреционным и мандатным управлением доступом, основные виды правил AppArmor, режимы применения политик, примеры конфигурации и особенности использования AppArmor в Linux.

2 Теоретические основы управления доступом

2.1 Дискреционное управление доступом

Дискреционная модель управления доступом, или **DAC (Discretionary Access Control)**, предполагает назначение разрешений в зависимости от пользователя, группы и прав на конкретный объект. В Linux базовые разрешения представлены символами `r` (чтение), `w` (запись) и `x` (выполнение или доступ к каталогу). Их можно просматривать командой `ls -l` и изменять, например, с помощью `chmod` и `chown`.

Если файл принадлежит определённому пользователю, его владелец в пределах предоставленных полномочий может изменять набор разрешений. Решение о доступе обычно зависит от учетных данных процесса и атрибутов объекта. Для многих повседневных операций данной модели достаточно, однако разрешения распространяются на процессы, действующие от имени соответствующего пользователя, без подробного разделения по назначению отдельных приложений.

Например, веб-службе может требоваться чтение файлов сайта из каталога `/var/www/`, но запуск службы от имени пользователя с доступом к домашнему каталогу не всегда позволяет средствами DAC отделить допустимую работу веб-сервера от обращения к посторонним файлам. Возникает потребность в

дополнительном механизме, который описывает границы полномочий самого приложения.

2.2 Мандатное управление доступом

MAC (Mandatory Access Control) — модель, в которой разрешения определяются централизованно установленной политикой безопасности. Приложение не может самостоятельно расширить установленные политикой полномочия только за счёт доступа своего пользователя к ресурсу. AppArmor относится к механизмам MAC, а его политика ориентирована прежде всего на ограничение поведения процессов [5].

В Linux проверки MAC дополняют традиционные проверки DAC. Для успешного выполнения действия недостаточно одного разрешения в профиле AppArmor: операция должна соответствовать также обычным правам пользователя и другим применимым ограничениям системы. Например, наличие правила AppArmor, разрешающего читать `/etc/example.conf`, не предоставляет приложению доступ к файлу, если файловая система запрещает чтение от имени пользователя процесса.

Принцип работы MAC тесно связан с **минимальными привилегиями**: программе следует предоставлять только доступ, необходимый для выполнения предусмотренных функций. Ограничение лишних полномочий уменьшает ущерб, который может возникнуть после эксплуатации уязвимости в приложении.

2.3 Назначение AppArmor в системе защиты Linux

AppArmor предназначен для ограничения приложений заранее определённым набором ресурсов. Для каждой защищаемой программы создаётся про-

филь, описывающий разрешённые операции с файлами, запуск других программ, использование отдельных возможностей ядра и сетевых механизмов. Если процесс пытается выполнить действие, отсутствующее в разрешающей части действующей политики, AppArmor может запретить его и зарегистрировать событие [6; 8].

При этом AppArmor не заменяет управление пользователями, обновление программного обеспечения, межсетевые экраны, резервное копирование и аудит. Его назначение заключается в добавлении независимого уровня контроля, который ограничивает последствия ошибок или компрометации приложений.

3 Архитектура и принцип работы AppArmor

3.1 Взаимодействие с Linux Security Modules

Linux Security Modules (LSM) — инфраструктура ядра Linux, предоставляющая точки проверки безопасности при выполнении системных операций. Через данные точки отдельные модули могут участвовать в принятии решений о доступе к ресурсам. AppArmor использует механизм LSM для применения правил к работающим процессам [5; 8].

В архитектуре AppArmor выделяются две основные части. Первая находится в пространстве ядра и непосредственно выполняет проверки доступа. Вторая располагается в пользовательском пространстве и включает файлы конфигурации, парсер политик и административные утилиты. Текстовые профили обрабатываются программой `apparmor_parser`, после чего подготовленная политика загружается в ядро.

Проверка доступа в общем виде состоит из следующих этапов:

1. Приложение обращается к ресурсу, например пытается открыть файл или создать сетевой сокет.
2. В ходе обработки операции ядро вызывает соответствующую точку проверки LSM.
3. AppArmor определяет контекст безопасности процесса и применяемый

к нему профиль.

4. Запрашиваемая операция сопоставляется с правилами действующей политики.
5. В зависимости от режима профиля доступ разрешается, отклоняется либо сопровождается диагностическим сообщением.

Проверка осуществляется на уровне ядра, поэтому обычное приложение не может отключить собственные ограничения простым изменением пользовательских настроек. Управление загруженными профилями требует соответствующих системных полномочий.

3.2 Профили приложений

Основной объект настройки AppArmor — **профиль безопасности**. Он представляет собой текстовое описание того, какие ресурсы разрешено использовать конкретному приложению. Профили обычно находятся в каталоге `/etc/apparmor.d/`. Их имена по принятой в ряде дистрибутивов традиции соотносятся с путями к исполняемым файлам, например `/etc/apparmor.d/usr.sbin.exampled` [6; 9].

Профиль может быть связан с исполняемым файлом по пути его запуска. После применения профиля процесс работает в ограниченном контексте, а дочерние процессы могут наследовать ограничения или переходить к другим профилям согласно правилам выполнения. При отсутствии применимого профиля процесс обычно работает в состоянии `unconfined`: специальные ограничения AppArmor для него не действуют, но обычные разрешения Linux сохраняются [8].

Из этого следует, что наличие установленного и запущенного AppArmor не означает автоматическую полную изоляцию всех программ. Степень защиты зависит от перечня охваченных процессов, качества профилей и их фактиче-

ского режима работы.

3.3 Управление доступом на основе путей

Характерной особенностью AppArmor является **модель управления доступом, основанная на путях файловой системы (path-based access control)**. В правилах указываются пути к объектам и допустимые операции, например `/etc/hosts r,` или `/var/log/example/** rw,`. Вместо обязательной ручной разметки каждого файла администратор описывает доступ через имена и шаблоны путей [3; 5].

Символ `*` используется для сопоставления с именами в пределах компонента пути, а `**` позволяет охватывать вложенные каталоги. Например, выражение `/srv/data/**` предназначено для файлов и объектов внутри дерева `/srv/data/`. При этом запись `/srv/data/` обозначает сам каталог и может потребовать отдельного разрешения.

Подход на основе путей облегчает чтение и сопровождение политик: администратор видит знакомую структуру файловой системы. Вместе с тем при изменении размещения приложений, использовании символических ссылок, перемещении данных или изменении схемы монтирования необходимо повторно проверять корректность профиля. Фактическая интерпретация пути зависит от возможностей и версии ядра, а также контекста файловой системы.

4 Структура политики безопасности AppArmor

4.1 Общий синтаксис профиля

Политика AppArmor описывается специальным декларативным языком. В начале файла могут подключаться стандартные переменные, после чего указывается имя профиля и, при необходимости, путь привязки к исполняемому файлу. Правила располагаются внутри фигурных скобок и обычно заканчиваются запятой [3; 9].

Упрощённый пример выглядит следующим образом:

```
#include <tunables/global>

profile example-app /usr/local/bin/example-app {
    #include <abstractions/base>

    /usr/local/bin/example-app mr,
    /etc/example-app/config.ini r,
    /var/lib/example-app/** rw,
}
```

Строка `#include <tunables/global>` подключает общие переменные путей.

Директива `profile example-app /usr/local/bin/example-app` задаёт имя профиля и путь, при запуске которого профиль применяется. Строка `#include <abstractions/base>` включает набор типовых разрешений, необходимых многим приложениям. Правило `/etc/example-app/config.ini r,` допускает чтение конфигурационного файла, а `/var/lib/example-app/** rw,` разрешает чтение и запись в указанном дереве объектов. Строка с `mr` разрешает чтение исполняемого файла и отображение его содержимого в исполняемую память.

Данный пример демонстрирует синтаксис, но не является универсальным готовым профилем. Реальная программа может требовать дополнительных правил для библиотек, устройств, каталогов, сетевого взаимодействия и запуска вспомогательных компонентов.

4.2 Правила доступа к файлам

Файловые разрешения AppArmor задаются буквенными обозначениями. К наиболее распространённым относятся:

- `r` — чтение содержимого файла;
- `w` — запись, включая создание и изменение файлов в рамках соответствующих правил;
- `a` — добавление данных в конец файла без обычного произвольного изменения содержимого;
- `m` — отображение файла в память с возможностью исполнения кода;
- `l` — создание жёстких ссылок с учётом дополнительных проверок;
- `k` — использование блокировки файла.

Например, правило `/etc/hostname r,` позволяет приложению читать файл с именем компьютера. Запись `/var/log/example.log a,` предоставляет разрешение добавлять сведения в журнал, а `deny /etc/shadow r,` явно запрещает

чтение базы хешей паролей. Явный запрет имеет приоритет над соответствующим разрешением; при этом правила `deny` обычно подавляют регистрацию совпавших отказов, если дополнительно не указан модификатор `audit` [4; 9].

Для исполнения программ используются специальные режимы переходов: `ix`, `px`, `Px`, `cx`, `Cx` и другие. Режим `ix` означает запуск с наследованием текущего профиля; `px` предполагает переход к отдельному профилю запускаемой программы, а `Px` дополнительно предусматривает очистку небезопасных переменных окружения. Режимы `cx` и `Cx` предназначены для переходов к дочерним профилям. Неправильный выбор режима запуска может привести к расширению области доступа или нарушению нормальной работы приложения, поэтому соответствующие правила требуют отдельного тестирования [9].

4.3 Контроль сетевого взаимодействия и полномочий

AppArmor может ограничивать создание и использование сетевых сокетов. В политике применяются правила `network`, позволяющие определять семейство адресов и тип сокета. Например:

```
network inet stream,
network inet6 stream,
network inet dgram,
```

Первая строка разрешает создание потоковых сокетов IPv4, вторая — потоковых сокетов IPv6, третья — дейтаграммных сокетов IPv4. Данные правила описывают категории сетевых операций и сами по себе не заменяют межсетевой экран. Уровень детализации сетевой политики зависит от поддерживаемой версии AppArmor и функциональности ядра [3; 5].

Отдельная группа ограничений связана с **Linux capabilities** — механизмом разделения привилегий суперпользователя на отдельные возможности. Например, строка `capability net_bind_service,` позволяет профилированному

процессу использовать соответствующую системную возможность при условии, что она предоставлена ему другими механизмами Linux. Запись в профиле не наделяет процесс полномочием, которого у него не было, а устанавливает дополнительное условие его использования.

Кроме того, AppArmor поддерживает правила для сигналов между процессами, операций `ptrace`, взаимодействия по D-Bus, сокетов Unix и отдельных действий с точками монтирования. С их помощью можно ограничивать не только файловый доступ, но и другие потенциально опасные функции программы [3; 5].

4.4 Абстракции, переменные и дочерние профили

При описании больших приложений возникает потребность повторно использовать типовые наборы разрешений. Для этого в AppArmor предусмотрены **абстракции (abstractions)**. Они представляют собой подключаемые фрагменты политики, содержащие общие правила для доступа к системным библиотекам, службам разрешения имён, пользовательским каталогам и другим распространённым ресурсам.

Например, директива `#include <abstractions/nameservice>` подключает правила, связанные с типовыми механизмами разрешения имён. Применение абстракций уменьшает дублирование и облегчает сопровождение политик. Вместе с тем каждая подключённая абстракция расширяет набор разрешений, поэтому включать её следует только при наличии реальной необходимости.

Переменные вида `@{HOME}` позволяют использовать заранее определённые наборы путей. Благодаря им один профиль проще адаптировать для различных системных конфигураций. При необходимости локальные дополнения могут размещаться в каталоге `/etc/apparmor.d/local/`, если пакетный

профиль содержит соответствующую директиву подключения [3; 6].

Для более сложных сценариев существуют дочерние профили и так называемые **hats** — специальные подчинённые области ограничения. Приложение, поддерживающее интерфейс `aa_change_hat`, может переключаться в более узкий контекст при выполнении определённого участка работы. Данный механизм используется, когда разные части одного приложения должны иметь разные уровни доступа [3].

5 Режимы работы и проверка политик

5.1 Режим enforce

В режиме **enforce (принудительное применение политики)** AppArmor проверяет операции приложения и блокирует действия, не разрешённые профилем. Например, если служба пытается прочитать файл, отсутствующий в перечне разрешённых ресурсов, ядро может вернуть ошибку доступа. Информация о запрете, в зависимости от настроек аудита, регистрируется в системных журналах [5; 6].

Режим `enforce` применяется для рабочих профилей, которые прошли проверку на соответствие нормальной функциональности приложения. Он обеспечивает непосредственное ограничение поведения процесса и является основным режимом защитной эксплуатации.

5.2 Режим complain

В режиме **complain (обучение и диагностика)** AppArmor преимущественно фиксирует операции, которые были бы запрещены активной политикой, но не блокирует их. Это помогает изучить реальные обращения приложения к ресурсам и постепенно составить корректный набор разрешений [6].

Например, при первоначальном составлении профиля службы администратор переводит профиль в `complain`, запускает предусмотренные рабочие сценарии и анализирует сформированные записи. После уточнения разрешений профиль переводится в `enforce` и проверяется повторно.

Следует учитывать, что режим `complain` не обеспечивает того же уровня изоляции, что `enforce`. Кроме того, явно заданные запрещающие правила и отдельные особенности обработки событий могут влиять на поведение политики. Поэтому данный режим предназначен прежде всего для подготовки и сопровождения профилей, а не для длительной эксплуатации критически важных служб.

5.3 Состояние unconfined

Состояние **unconfined** означает отсутствие действующего ограничивающего профиля AppArmor для процесса. При этом доступ по-прежнему определяется стандартными механизмами Linux и другими включёнными средствами защиты. В частности, нельзя считать, что все программы в системе защищены только потому, что служба AppArmor работает и часть профилей находится в режиме `enforce` [8].

Администратор должен регулярно проверять как общее состояние подсистемы, так и перечень процессов, оставшихся без профилей.

6 Настройка и администрирование AppArmor в Linux

6.1 Установка и проверка состояния

В Ubuntu AppArmor обычно устанавливается и активируется по умолчанию. Для проверки доступности модуля и состояния профилей можно использовать команды [6]:

```
sudo systemctl status apparmor
sudo aa-status
```

Команда `systemctl status apparmor` отображает состояние соответствующей службы загрузки политик. Утилита `aa-status` показывает сведения о работающем механизме AppArmor, числе загруженных профилей, их режимах и некоторых ограниченных процессах. Для просмотра набора действующих модулей LSM может использоваться команда:

```
cat /sys/kernel/security/lsm
```

На системах семейства Debian и Ubuntu необходимые административные инструменты устанавливаются командой:

```
sudo apt install apparmor apparmor-utils
```

Работа команд и доступность пакетов зависят от дистрибутива, версии ядра и состава репозиториев. Сам факт запуска `apparmor.service` следует оценивать совместно с результатами `aa-status`, поскольку служба в первую очередь отвечает за загрузку политик, а проверки выполняются в ядре.

6.2 Создание и загрузка профиля

В качестве примера рассмотрим условную программу `/usr/local/bin/report-reader`, которая читает входные данные из `/var/lib/report-reader/input/`, использует конфигурационный файл `/etc/report-reader/config.ini` и записывает журнал в `/var/log/report-reader/`. Для ограничения её доступа можно подготовить файл `/etc/apparmor.d/usr.local.bin.report-reader` со следующим содержимым:

```
#include <tunables/global>

profile report-reader /usr/local/bin/report-reader {
    #include <abstractions/base>

    /usr/local/bin/report-reader mr,
    /etc/report-reader/config.ini r,
    /var/lib/report-reader/input/ r,
    /var/lib/report-reader/input/** r,
    /var/log/report-reader/ rw,
    /var/log/report-reader/** rw,

    deny /etc/shadow r,
```

```
}
```

В профиле разрешено чтение конфигурации и входных данных, а запись ограничена каталогом журналов. Явный запрет для `/etc/shadow` дополнительно демонстрирует возможность исключать критически важные объекты. Здесь не указаны разрешения на создание сетевых соединений IPv4/IPv6; фактический состав остальных допусков определяется в том числе подключённой абстракцией `base`.

После сохранения файла профиль можно загрузить или обновить командой:

```
sudo apparmor_parser -r /etc/apparmor.d/usr.local.bin.report-reader
```

Параметр `-r` используется для замены загруженной политики актуальной версией. Для повторной загрузки системных профилей в Ubuntu также применяется команда `sudo systemctl reload apparmor`. После внесения изменений необходимо убедиться, что профиль успешно загружен и программа продолжает выполнять предусмотренные операции [6].

Примечание: пример является учебным. Исполняемый файл, пути данных и набор разрешений должны соответствовать фактической структуре установленного приложения. Представленный профиль не запускался в рамках подготовки данного реферата, поэтому работоспособность конкретной программы не утверждается.

6.3 Формирование профилей на основе наблюдений

Создание правил вручную требует изучения работы приложения. Для облегчения процедуры в набор AppArmor входят утилиты `aa-genprof` и `aa-logprof`. Первая помогает создать новый профиль на основе наблюдаемых

обращений к ресурсам, а вторая анализирует журнал событий и предлагает уточнить существующие разрешения [2; 6].

Пример вызова генератора:

```
sudo aa-genprof /usr/local/bin/report-reader
```

После запуска приложения с предусмотренными рабочими сценариями результаты анализа можно использовать для редактирования профиля:

```
sudo aa-logprof
```

Переключение режимов выбранного профиля выполняется следующими командами:

```
sudo aa-complain /usr/local/bin/report-reader
sudo aa-enforce /usr/local/bin/report-reader
```

Первая команда полезна для изучения потенциальных ограничений, вторая — для перехода к принудительному применению политики. Предлагаемые автоматическими утилитами разрешения необходимо проверять вручную: наблюдение за приложением фиксирует фактически выполненные операции, но не определяет, все ли они безопасны и необходимы.

6.4 Диагностика отказов и анализ журналов

В ходе эксплуатации некорректное правило может помешать приложению открыть файл, подключиться к сокету или выполнить вспомогательную программу. Сведения о соответствующих событиях можно искать в журнале ядра, системном журнале и файлах аудита. Например:

```
sudo journalctl -k | grep 'apparmor='
sudo grep 'apparmor=' /var/log/audit/audit.log
```

Вторая команда применима при наличии соответствующего файла и настроенного аудита. Диагностическая запись AppArmor может содержать имя профиля, исполняемый файл, путь ресурса, запрошенное разрешение и результат проверки. Значение `apparmor="DENIED"` обычно указывает на отклонённую операцию, а `apparmor="ALLOWED"` может встречаться при регистрации действий в режиме обучения или расширенного аудита [1; 4].

При выявлении отказа сначала следует определить, действительно ли приложению необходим запрещённый ресурс. Если операция соответствует его рабочим функциям, профиль уточняется минимальным разрешением. Если обращение выглядит посторонним или подозрительным, добавлять разрешение без выяснения причин недопустимо.

7 Применение AppArmor для защиты приложений

7.1 Защита серверных служб

Одним из наиболее значимых направлений использования AppArmor является защита сетевых демонов и фоновых служб. Веб-серверу могут требоваться чтение файлов сайта, обращение к сертификатам, работа с определёнными сокетами и запись журналов. При этом ему не должен предоставляться произвольный доступ к пользовательским каталогам, служебным ключам или исполняемым файлам, не связанным с его задачами.

Профиль AppArmor позволяет описать необходимые ресурсы и ограничить последствия эксплуатации уязвимости в серверном процессе. Например, при попытке уязвимой программы прочитать посторонний конфиденциальный файл запрос может быть отклонён, даже если обычные права пользователя допускают чтение.

Однако безопасность зависит от точности правил. Чрезмерно широкий шаблон вроде `/** rw,` практически лишает профиль смысла в отношении файлового доступа. Поэтому для производственных служб желательно разрешать отдельные каталоги и операции, а также исключать ненужные способы запуска дочерних процессов.

7.2 Рабочие станции, контейнеры и изоляция процессов

AppArmor применяется не только на серверах, но и в пользовательских системах. Ограничение браузеров, обработчиков документов, графических программ и других приложений позволяет снижать риск обращения к ресурсам, не связанным с их назначением. В Ubuntu механизм AppArmor также используется в системе безопасности пакетов Snap [5].

В контейнерных средах профили AppArmor могут служить дополнительным средством ограничения процессов. При этом контейнеризация сама по себе не отменяет необходимости настройки файловых прав, пространств имён, Linux capabilities и других механизмов защиты. Конкретный способ назначения профиля контейнеру зависит от используемой платформы и её конфигурации.

Совместное применение AppArmor с изоляцией процессов, ограничением системных вызовов через `seccomp` и управлением привилегиями формирует несколько независимых рубежей безопасности. Если один механизм не предотвращает определённое действие, дополнительный уровень контроля может уменьшить возможности атакующего.

8 Сравнение моделей AppArmor и SELinux

Наряду с AppArmor в Linux широко применяется **SELinux (Security-Enhanced Linux)** — механизм мандатного управления доступом, также использующий LSM. Оба решения предназначены для ограничения действий программ сверх обычных прав файловой системы, однако отличаются способом представления политики и организации контекстов безопасности [5; 7].

Критерий	AppArmor	SELinux
Основной подход	Политики, ориентированные на приложения и пути к ресурсам	Политики, основанные на контекстах безопасности и метках объектов
Описание файлового доступа	Шаблоны абсолютных путей файловой системы	Типы и метки безопасности, назначаемые объектам
Организация ограничений	Профили приложений и правила доступа к ресурсам	Домены процессов, типы объектов, правила переходов и разрешений

Критерий	AppArmor	SELinux
Настройка	Часто проще для отдельного приложения с известной структурой каталогов	Требует понимания типов, контекстов и принятой системной политики
Типичная область применения	Поэтапное ограничение отдельных программ и сервисов	Комплексное управление доменами и взаимодействием системных компонентов
Режим обучения	`complain`	`permissive`

Главное практическое отличие заключается в организации правил. AppArmor связывает ограничения с профилем приложения и использует понятные администратору пути. SELinux использует метки безопасности, благодаря которым контроль может быть менее зависим от конкретного расположения файла в дереве каталогов, но настройка соответствующих контекстов требует дополнительных знаний.

Нельзя считать один механизм безусловно безопаснее другого. Эффективность определяется корректностью политики, полнотой её применения, требованиями инфраструктуры и квалификацией администратора. Для ограничения отдельных служб удобным выбором часто оказывается AppArmor; для систем, где уже используется развитая политика SELinux, может быть рациональнее поддерживать её без смены архитектуры безопасности.

9 Преимущества, ограничения и рекомендации по использованию

9.1 Преимущества AppArmor

К основным достоинствам AppArmor относятся:

1. **Относительная наглядность правил.** Использование путей к файлам позволяет быстро определить, какие ресурсы доступны программе.
2. **Дополнительный контроль сверх DAC.** Профиль продолжает ограничивать приложение, даже если пользователь процесса имеет более широкие файловые разрешения.
3. **Поэтапное внедрение.** Можно начать с наиболее критичных служб и постепенно увеличивать число ограниченных приложений.
4. **Поддержка режима обучения.** Режим `complain` и утилиты анализа журналов помогают выявлять необходимые разрешения.
5. **Гибкость ограничений.** Помимо файловых правил можно контролировать запуск программ, отдельные возможности ядра и различные способы межпроцессного взаимодействия.
6. **Интеграция с Linux.** AppArmor поддерживается ядром и средствами администрирования ряда распространённых дистрибутивов [3; 5; 6; 8; 9].

9.2 Ограничения и возможные сложности

Основным ограничением AppArmor является зависимость реальной защиты от качества и полноты профилей. Процесс без загруженного профиля остаётся в состоянии `unconfined`, а ошибочно разрешённые действия могут использоваться при атаке. Наличие механизма MAC не устраняет уязвимости приложений и не гарантирует конфиденциальность всех данных.

Дополнительную сложность создаёт привязка правил к путям. Перенос приложения, изменение структуры каталогов, переименование ресурсов и особенности монтирования могут потребовать корректировки политики. Также нужно учитывать, что разрешения для многокомпонентных программ с динамически загружаемыми библиотеками, вспомогательными процессами и сетевым взаимодействием нередко оказываются сложнее первоначальных ожиданий.

Возможны и ложные блокировки: при недостаточно полном профиле нормальная операция будет отклонена. Для предотвращения подобных ситуаций необходимы предварительное тестирование, анализ журналов и регулярная актуализация конфигурации после обновлений программного обеспечения.

9.3 Рекомендации по безопасной эксплуатации

При внедрении AppArmor целесообразно придерживаться следующего порядка:

1. Определить наиболее важные приложения и ресурсы, доступ к которым требуется ограничить.
2. Изучить существующие профили, служебные зависимости и обычные сценарии работы выбранной программы.

3. Составить минимальный набор разрешений, избегая неоправданно широких шаблонов файлов и сетевых правил.
4. Проверить новую политику в тестовой среде, используя режим `complain` для сбора необходимой диагностической информации.
5. Перевести профиль в `enforce`, проверить запуск, остановку, перезапуск и основные операции приложения.
6. Анализировать записи об отказах и вносить изменения только после подтверждения необходимости соответствующего доступа.
7. Хранить изменения политик под контролем версий и пересматривать профили после обновления приложений и изменения инфраструктуры.

Выполнение перечисленных рекомендаций позволяет поддерживать баланс между работоспособностью сервисов и ограничением их потенциально опасных полномочий.

10 Заключение

В ходе подготовки реферата рассмотрена модель управления доступом AppArmor, предназначенная для дополнительной защиты процессов в операционной системе Linux. Установлено, что AppArmor реализует мандатный контроль доступа через механизм Linux Security Modules и дополняет традиционную дискреционную модель разрешений пользователей и файловых систем.

Изучены принципы применения профилей безопасности, основанных на путях к ресурсам, основные типы правил и возможности ограничения файлового доступа, сетевого взаимодействия, исполнения программ и отдельных привилегий ядра. Рассмотрены режимы `enforce` и `complain`, а также состояние `unconfined`, которое необходимо учитывать при оценке полноты защиты.

Отдельное внимание уделено администрированию AppArmor: проверке состояния подсистемы, созданию и загрузке профилей, использованию утилит `aa-genprof`, `aa-logprof`, `aa-complain` и `aa-enforce`, а также анализу отказов в системных журналах. На учебном примере продемонстрирован синтаксис профиля, ограничивающего чтение конфигурации, доступ к рабочим данным и запись служебных журналов.

Сравнение с SELinux показало, что AppArmor отличается преимущественно прикладной ориентацией и описанием файловых разрешений через пути, что может упрощать внедрение для отдельных приложений. Одновременно эффективность механизма зависит от актуальности и точности профилей, а отсутствие политики для процесса не обеспечивает его изоляцию средства-

ми AppArmor.

Следовательно, AppArmor представляет собой важный компонент многоуровневой защиты Linux, позволяющий реализовать принцип минимальных привилегий и уменьшить последствия компрометации приложений. Наибольшую практическую пользу его использование приносит при сочетании с корректным управлением пользователями, своевременными обновлениями программ, мониторингом событий безопасности и регулярной проверкой действующих политик.

11 Список литературы

1. *AppArmor Project*. Audit the System. — URL: https://wiki.apparmor.net/monitoring/AppArmorAuditing/ (visited on 10/24/2026).
2. *AppArmor Project*. Generate Profiles with aa-genprof. — URL: https://wiki.apparmor.net/getting-started/generate-with-aa-genprof/ (visited on 10/24/2026).
3. *AppArmor Project*. Profiles Quick Reference. — URL: https://wiki.apparmor.net/reference/profiles-quick-reference/ (visited on 10/24/2026).
4. *AppArmor Project*. Troubleshooting. — URL: https://wiki.apparmor.net/monitoring/AppArmor_Failures/ (visited on 10/24/2026).
5. *Canonical*. AppArmor — Ubuntu Security Documentation. — URL: https://documentation.ubuntu.com/security/security-features/privilege-restriction/apparmor/ (visited on 10/24/2026).
6. *Canonical*. AppArmor — Ubuntu Server Documentation. — URL: https://ubuntu.com/server/docs/how-to/security/apparmor/ (visited on 10/24/2026).
7. *Canonical*. Privilege Restriction — Ubuntu Security Documentation. — URL: https://documentation.ubuntu.com/security/security-features/privilege-restriction/ (visited on 10/24/2026).
8. *Linux Kernel Documentation*. AppArmor — Linux Security Modules. — URL: https://docs.kernel.org/admin-guide/LSM/apparmor.html (visited on 10/24/2026).

9. *Ubuntu Manpages*. apparmor.d(5) — Syntax of Security Profiles for AppArmor. — URL: https://manpages.ubuntu.com/manpages/jammy/man5/apparmor.d.5.html (visited on 10/24/2026).